1. ABSTRACT

The project titled **"Real-Time DevSecOps BINGO Application – Infrastructure with Terraform and Jenkins CI/CD Pipeline"** demonstrates the implementation of a real-time multiplayer web application together with an automated DevSecOps pipeline.

The BINGO application is developed using **Next.js, React and Socket.IO**. Socket.IO enables real-time communication between multiple players connected to the same game room.

The infrastructure is hosted on **Amazon Web Services (AWS)** using an EC2 instance. Terraform is used as Infrastructure as Code to provision the required cloud resources.

A Jenkins-based CI/CD pipeline automates the software delivery process. The pipeline integrates **SonarQube** for source-code quality analysis, **OWASP Dependency-Check** for dependency vulnerability scanning, **Trivy** for security scanning, and **Docker** for containerization.

The overall workflow starts with source code stored in GitHub and continues through code checkout, quality analysis, dependency scanning, filesystem scanning, Docker image creation, container image scanning and deployment.

The project demonstrates how security can be integrated into CI/CD rather than being performed only after deployment.

The reference project identifies the BINGO application as a multiplayer application developed with Next.js, React and Socket.IO.

---

2. INTRODUCTION

Modern software applications require fast development, reliable deployment and continuous security validation.

Traditional software development often separates development, operations and security activities. This can result in security issues being discovered late in the development lifecycle.

DevSecOps addresses this problem by integrating security practices directly into the CI/CD pipeline.

This project combines:

Development
↓
GitHub
↓
Jenkins

↓
Code Quality
↓
Security Scanning
↓
Docker
↓
Docker Hub
↓
AWS EC2
↓
Application

The project demonstrates the complete lifecycle of a real-time web application from source-code management to cloud deployment.

---

3. PROBLEM STATEMENT

Developing and deploying applications manually can introduce several problems:

- Manual deployment errors
- Lack of automated testing
- Security vulnerabilities in dependencies
- Poor source-code quality visibility
- Inconsistent deployment environments
- Manual infrastructure configuration
- Difficulty reproducing infrastructure
- Lack of automated security checks

Therefore, the objective is to implement an automated DevSecOps pipeline that integrates infrastructure provisioning, continuous integration, security scanning, containerization and deployment.

---

4. PROJECT OBJECTIVES

The main objectives are:

1. Develop a real-time multiplayer BINGO application.
2. Implement real-time communication using Socket.IO.
3. Store application source code in GitHub.
4. Provision AWS infrastructure using Terraform.
5. Deploy Jenkins on AWS EC2.
6. Configure Jenkins for CI/CD.
7. Integrate SonarQube.
8. Implement a SonarQube Quality Gate.
9. Integrate OWASP Dependency-Check.

10. Integrate Trivy.
11. Containerize the application using Docker.
12. Push Docker images to Docker Hub.
13. Scan Docker images using Trivy.
14. Deploy the application to AWS EC2.
15. Validate the complete CI/CD workflow.

The reference project specifically lists Terraform provisioning, Jenkins, SonarQube Scanner, Node.js, OWASP Dependency-Check and Docker-related configuration as project steps.

---

5. PROJECT SCOPE

5.1 Application Scope

The application provides:

- Room creation
- Room joining
- Player names
- Multiplayer interaction
- BINGO board
- Number calling
- Real-time updates
- Basic chat functionality
- Game reset

The reference application demonstrates entering a Room ID and player name and shows a server-side record of players joining a room.

5.2 DevOps Scope

The project includes:

- Git
- GitHub
- Terraform
- AWS EC2
- Jenkins
- Docker
- Docker Hub

5.3 Security Scope

Security is implemented through:

- SonarQube
- OWASP Dependency-Check
- Trivy filesystem scan
- Trivy Docker image scan

---

6. EXISTING SYSTEM

In a traditional deployment process:

Developer
↓
Manual Build
↓
Manual Testing
↓
Manual Deployment

Problems include:

- Manual intervention
- Higher possibility of errors
- Security checks performed late
- Infrastructure configured manually
- Difficult rollback/reproduction
- Slow deployment process

---

7. PROPOSED SYSTEM

The proposed system automates the complete workflow:

Developer
↓
GitHub
↓
Jenkins
↓
SonarQube
↓
Quality Gate
↓
OWASP Dependency-Check
↓
Trivy Filesystem Scan
↓
Docker Build
↓
Docker Hub
↓

Trivy Image Scan
↓
Deployment
↓
AWS EC2
↓
BINGO Application

The reference project adds OWASP and Trivy stages after the initial Jenkins/SonarQube pipeline and then adds Docker build/push and image scanning.

---

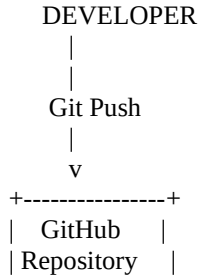
8. TECHNOLOGIES USED

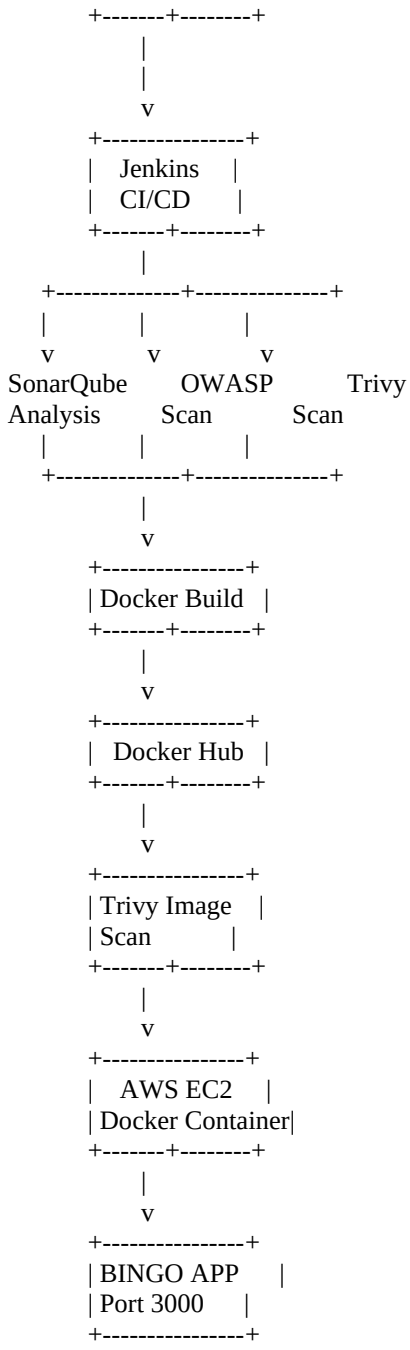
Technology	Purpose
Next.js	Web application framework
React	Frontend user interface
Socket.IO	Real-time communication
Node.js	Backend/runtime
Git	Version control
GitHub	Source-code repository
AWS	Cloud platform
EC2	Cloud server
Terraform	Infrastructure as Code
Jenkins	CI/CD automation
SonarQube	Code quality analysis
OWASP Dependency-Check	Dependency security analysis
Trivy	Vulnerability scanning
Docker	Containerization
Docker Hub	Container registry
MobaXterm	SSH access

---

9. SYSTEM ARCHITECTURE

Use this architecture diagram in your document:





Screenshot to add

Screenshot 1 – Overall Architecture Diagram

Create this yourself in:

- draw.io
- PowerPoint
- Canva
- Lucidchart

Caption:

Figure 1: Overall DevSecOps architecture of the Real-Time BINGO application

---

10. APPLICATION ARCHITECTURE

The application consists of two major parts.

Frontend

Built using:

- Next.js
- React
- CSS

The frontend provides:

- Room ID input
- Player name input
- BINGO board
- Called numbers
- Chat
- Player information

Backend

The backend uses:

- Node.js
- Socket.IO

Socket.IO manages real-time events between players.

Example:

```
Player 1
   |
   | Call Number
   v
Socket.IO Server
   |
   +------------+
   |            |
   v            v
Player 2      Player 3
```

---

11. APPLICATION FEATURES

11.1 Create Room

A user can create a BINGO room.

11.2 Join Room

Other users can enter the Room ID and join.

11.3 Multiplayer

Multiple players can participate in the same room.

11.4 Real-Time Number Calling

When a number is called, connected players receive the update.

11.5 Chat

Players can communicate using the room chat.

11.6 BINGO Board

Each player receives a BINGO board.

11.7 Reset

The game can be reset.

Screenshots to add

Screenshot 2 – BINGO Landing Page

Show:

- Room ID
- Your Name
- Enter
- Create Room

The reference PDF includes this screen.

Caption:

Figure 2: BINGO application landing page

Screenshot 3 – Player Joined Room

Show the application after joining a room.

Caption:

Figure 3: Player joining a BINGO room

Screenshot 4 – Multiplayer Room

Open two browser windows and show both players.

Caption:

Figure 4: Multiple players connected to the same BINGO room

Screenshot 5 – BINGO Game Board

Show the actual BINGO board.

Caption:

Figure 5: BINGO game board and called numbers

Screenshot 6 – Chat

Show players exchanging messages.

Caption:

Figure 6: Real-time chat between players

---

12. PROJECT DIRECTORY STRUCTURE

Add this:

```
bingo-devsecops-project/
├── pages/
```

```
    ├── index.js
    └── _app.js

├── styles/
    └── globals.css

├── server.js
├── package.json
├── Dockerfile
├── Jenkinsfile
├── sonar-project.properties
├── .gitignore
├── .dockerignore
├── README.md

├── terraform/
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    ├── terraform.tfvars.example
    └── user-data.sh

└── scripts/
    ├── local-test.sh
    ├── check-server.sh
    └── deploy-manual.sh
```

Screenshot to add

Screenshot 7 – VS Code Project Structure

Open the project in VS Code and expand the folders.

Caption:

Figure 7: BINGO DevSecOps project source-code structure

---

13. GITHUB REPOSITORY

GitHub is used to store and manage the source code.

The workflow is:

```
Local Project
     ↓
git add
     ↓
git commit
     ↓
git push
     ↓
GitHub
```

Example:

```
git init
git branch -M main
git add .
git commit -m "Initial BINGO DevSecOps project"
git push -u origin main
```

Screenshot to add

Screenshot 8 – GitHub Repository

Show your actual repository containing:

```
Dockerfile
Jenkinsfile
package.json
server.js
pages
styles
terraform
```

Do not show tokens/secrets.

Caption:

Figure 8: BINGO application source code repository in GitHub

---

14. LOCAL APPLICATION SETUP

Before deploying to AWS, the application was tested locally.

Install dependencies:

```
npm install
```

Build:

```
npm run build
```

Start:

```
npm start
```

Application:

```
http://localhost:3000
```

The reference project also demonstrates local testing and a local BINGO URL.

Screenshot to add

Screenshot 9 – Local Application Running

Show browser:

http://localhost:3000

Caption:

Figure 9: BINGO application running successfully in the local environment

---

15. AWS INFRASTRUCTURE

AWS EC2 is used as the cloud server.

The EC2 instance hosts the DevSecOps tools and application.

Required ports:

Port	Service
22	SSH
3000	BINGO
8080	Jenkins
9000	SonarQube

The reference project validates Jenkins through the instance IP and port 8080 and starts SonarQube on port 9000.

Screenshot to add

Screenshot 10 – AWS EC2 Instance

Show:

- Instance running
- Instance type
- Public IPv4
- Security group

Hide sensitive information where appropriate.

Caption:

Figure 10: AWS EC2 instance used for the DevSecOps environment

16. SECURITY GROUP

The security group allows the required application and administration traffic.

Example:

SSH → 22
BINGO → 3000
Jenkins → 8080
SonarQube → 9000

For better security, SSH should preferably be restricted to your own public IP.

Screenshot 11 – Security Group

Show inbound rules.

Caption:

Figure 11: AWS EC2 security group configuration

---

17. TERRAFORM INFRASTRUCTURE AS CODE

Terraform is used to automate infrastructure provisioning.

Terraform removes the need to manually create every infrastructure component.

Basic workflow:

Terraform Code
↓
terraform init
↓
terraform validate
↓
terraform plan
↓
terraform apply
↓
AWS Resources

The reference project uses:

terraform apply

and:

```
terraform apply --auto-approve
```

for provisioning.

Terraform files

```
terraform/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars.example
└── user-data.sh
```

Screenshot 12 – Terraform Init

Show terminal:

```
terraform init
```

Caption:

Figure 12: Terraform initialization

Screenshot 13 – Terraform Plan

Show:

```
terraform plan
```

Caption:

Figure 13: Terraform infrastructure plan

Screenshot 14 – Terraform Apply

Show successful:

```
Apply complete!
```

The reference document also shows successful resource creation after `terraform apply`.

Caption:

Figure 14: Successful Terraform infrastructure provisioning

---

18. EC2 SERVER CONFIGURATION

The server was configured with the required DevSecOps tools.

Installed components include:

Java
Jenkins
Docker
SonarQube
Trivy
Terraform
Node.js
AWS CLI
kubectl
eksctl

The reference project's installation script includes Jenkins with Java 17, Docker, SonarQube, Trivy, Terraform, kubectl, AWS CLI and eksctl.

Verification

java --version
node --version
npm --version
docker --version
trivy --version
terraform -version
kubectl version --client
aws --version
eksctl version

The reference project finishes with similar version-validation commands.

Screenshot 15 – Tool Versions

Show the terminal with all installed versions.

Caption:

Figure 15: Verification of DevSecOps tools installed on the EC2 instance

---

19. JENKINS CONFIGURATION

Jenkins is used as the CI/CD automation server.

The Jenkins workflow consists of:

GitHub
↓
Checkout
↓
Build

↓
Security Checks
↓
Docker
↓
Deployment

The reference project installs Jenkins and retrieves the initial administrator password using:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Screenshot 16 – Jenkins Dashboard

Show your Jenkins dashboard.

Caption:

Figure 16: Jenkins dashboard

Screenshot 17 – Jenkins Plugins

Show installed plugins such as:

- Eclipse Temurin
- SonarQube Scanner
- NodeJS
- OWASP Dependency-Check
- Docker-related plugins

The reference document lists these plugin categories.

Caption:

Figure 17: Jenkins plugins configured for the DevSecOps pipeline

---

20. JENKINS GLOBAL TOOLS

Jenkins Global Tool Configuration contains the required tools.

Examples:

```
JDK 17
Node.js
SonarQube Scanner
OWASP Dependency-Check
Docker
```

The reference project configures these under **Manage Jenkins → Tools**.

Screenshot 18

Show:

Manage Jenkins
↓
Tools

Caption:

Figure 18: Jenkins Global Tool Configuration

---

21. SONARQUBE

SonarQube is used to analyze source code.

It helps identify:

- Bugs
- Code smells
- Vulnerabilities
- Maintainability problems
- Security hotspots

The reference project creates a SonarQube project and integrates its token with Jenkins.

SonarQube workflow

Jenkins
↓
SonarQube Scanner
↓
SonarQube Server
↓
Analysis
↓
Quality Gate

Screenshot 19 – SonarQube Dashboard

Show your project.

Caption:

Figure 19: SonarQube project dashboard

Screenshot 20 – SonarQube Quality Gate

Show:

Quality Gate
Passed

if that is what your completed project produced.

The reference project includes a screenshot showing a passed Quality Gate.

Caption:

Figure 20: SonarQube Quality Gate result

⚠️ Important

Do **not** include your SonarQube token in the screenshot.

If the token appears, blur it.

---

22. SONARQUBE JENKINS INTEGRATION

The Jenkins pipeline performs SonarQube analysis.

Conceptually:

stage("Sonarqube Analysis")
↓
SonarQube Scanner
↓
SonarQube Server

The reference pipeline uses a SonarQube environment and scanner with a project name and project key.

---

23. QUALITY GATE

The Quality Gate determines whether the analyzed code meets the configured quality criteria.

Workflow:

Source Code
↓
SonarQube Analysis

↓
Quality Gate
↓
Passed / Failed

The reference pipeline uses waitForQualityGate after the SonarQube analysis.

Screenshot 21 – Jenkins Quality Gate Stage

Show the Jenkins Stage View with:

SonarQube Analysis
Quality Gate

Caption:

Figure 21: Jenkins SonarQube analysis and Quality Gate stages

---

24. OWASP DEPENDENCY-CHECK

OWASP Dependency-Check is used to identify known vulnerabilities in third-party dependencies.

The reference project explains that Dependency-Check analyzes libraries, frameworks and components against known vulnerability databases and can be integrated into CI/CD.

Workflow

package.json
↓
Dependencies
↓
OWASP Dependency-Check
↓
Vulnerability Report

The reference pipeline includes:

OWASP FS SCAN

and publishes the dependency-check report.

Screenshot 22 – OWASP Results

Show your Jenkins Dependency-Check results.

Caption:

Figure 22: OWASP Dependency-Check vulnerability report

If vulnerabilities appear, **do not hide them**. Explain that the scanner identified them and document the remediation/assessment.

---

25. TRIVY

Trivy is an open-source security scanner.

The reference project describes Trivy as a vulnerability scanner used for containerized environments and DevSecOps pipelines.

Two scans are performed.

25.1 Trivy Filesystem Scan

Source Code
↓
Trivy FS
↓
Security Findings

Reference pipeline:

```
trivy fs . > trivyfs.txt
```

Screenshot 23 – Trivy Filesystem Scan

Caption:

Figure 23: Trivy filesystem security scan

---

26. TRIVY DOCKER IMAGE SCAN

After creating the Docker image:

Docker Image
↓
Trivy
↓
Vulnerability Report

Reference pipeline:

```
trivy image YOUR_IMAGE:latest
```

The reference project adds a Trivy image scan after Docker build and push.

Screenshot 24 – Trivy Image Scan

Show your actual result.

Caption:

Figure 24: Trivy Docker image vulnerability scan

---

27. DOCKER

Docker is used to package the BINGO application.

Docker provides consistency between development and deployment environments.

Docker workflow

Source Code
↓
Dockerfile
↓
Docker Build
↓
Docker Image
↓
Docker Hub
↓
EC2
↓
Container

---

28. DOCKERFILE

The project contains a:

Dockerfile

The Dockerfile defines how the BINGO application is packaged into a Docker image.

Basic process:

Base Image
↓
Copy Application
↓
Install Dependencies
↓
Build Application
↓

Expose Port
↓
Start Application

Screenshot 25 – Dockerfile

Open the Dockerfile in VS Code.

Caption:

Figure 25: Dockerfile used to containerize the BINGO application

---

29. DOCKER BUILD

Example:

```
docker build -t bingo .
```

The image can be verified:

```
docker images
```

Screenshot 26 – Docker Build

Caption:

Figure 26: Successful Docker image build

---

30. DOCKER HUB

Docker Hub is used as the container registry.

Workflow:

Jenkins
↓
Docker Build
↓
Docker Tag
↓
Docker Push
↓
Docker Hub

The reference project uses Docker Hub credentials in Jenkins for Docker build/push and later pull/deployment.

Screenshot 27 – Docker Hub Repository

Show:

bingo-app
latest

Caption:

Figure 27: BINGO Docker image stored in Docker Hub

Screenshot 28 – Jenkins Docker Credential

Show the Jenkins credential configuration, but **hide the password/token**.

Caption:

Figure 28: Docker Hub credential configured securely in Jenkins

---

31. JENKINS DOCKER BUILD AND PUSH

The pipeline performs:

Docker Build
↓
Docker Tag
↓
Docker Push

The reference pipeline contains a Docker Build & Push stage and pushes the tagged image to Docker Hub.

Screenshot 29 – Docker Build & Push Stage

Show Jenkins.

Caption:

Figure 29: Jenkins Docker Build and Push stage

---

32. JENKINSFILE

The Jenkinsfile defines the automated pipeline.

Example stages:

Clean Workspace
↓
Checkout
↓
SonarQube Analysis
↓
Quality Gate
↓
Install Dependencies
↓
OWASP Scan
↓
Trivy Filesystem Scan
↓
Build
↓
Docker Build
↓
Docker Push
↓
Trivy Image Scan
↓
Deploy

The reference Jenkins pipeline begins with workspace cleanup and Git checkout, then performs SonarQube analysis and a Quality Gate.

Screenshot 30 – Jenkinsfile

Show your Jenkinsfile in GitHub or VS Code.

Caption:

Figure 30: Jenkinsfile defining the DevSecOps CI/CD pipeline

---

33. COMPLETE CI/CD PIPELINE

The complete pipeline is:

GitHub
|
v
Jenkins
|
v
Clean Workspace
|
v
Git Checkout
|
v

Install Dependencies
|
v
SonarQube Analysis
|
v
Quality Gate
|
v
OWASP Dependency Check
|
v
Trivy FS Scan
|
v
Application Build
|
v
Docker Build
|
v
Docker Push
|
v
Trivy Image Scan
|
v
Deploy
|
v
AWS EC2
|
v
BINGO Application

---

34. JENKINS PIPELINE RESULT

This is one of the **most important screenshots in your entire documentation**.

Your Jenkins Stage View should show stages such as:

Declarative Tool Install
Clean Workspace
Checkout from Git
SonarQube Analysis
Quality Gate
Install Dependencies
OWASP FS Scan
Trivy FS Scan
Docker Build & Push
Trivy
Deploy

The reference project contains a Jenkins build screenshot showing multiple pipeline stages and a Dependency-Check trend.

Screenshot 31 – FINAL JENKINS PIPELINE

Caption:

Figure 31: Successful end-to-end Jenkins CI/CD pipeline

Make this screenshot large.

This should probably be the **main result screenshot** in your report.

---

35. DEPLOYMENT TO EC2

After the Docker image is available:

```
Docker Hub
   ↓
docker pull
   ↓
EC2
   ↓
docker run
   ↓
BINGO
```

Example:

```
docker pull YOUR_DOCKERHUB_USERNAME/bingo-app:latest
```

Run:

```
docker run -d \
  --name bingo-app \
  -p 3000:3000 \
  YOUR_DOCKERHUB_USERNAME/bingo-app:latest
```

Verify:

```
docker ps
```

---

36. APPLICATION DEPLOYMENT VALIDATION

Open:

```
http://YOUR_EC2_PUBLIC_IP:3000
```

Screenshot 32 – Live BINGO Application

This is another **very important screenshot**.

Show the browser with your EC2-hosted BINGO application.

Caption:

Figure 32: BINGO application successfully deployed on AWS EC2

---

37. DOCKER CONTAINER VALIDATION

Run:

```
docker ps
```

You should see your BINGO container.

Screenshot 33

Show:

```
CONTAINER ID
IMAGE
STATUS
PORTS
NAMES
```

Caption:

Figure 33: BINGO Docker container running successfully on AWS EC2

---

38. MULTIPLAYER VALIDATION

Open the application from two browser sessions.

Example:

```
Player 1
Room: demo

Player 2
Room: demo
```

Both should join the same room.

Screenshot 34 – Multiplayer

Show both players.

Caption:

Figure 34: Real-time multiplayer BINGO functionality

---

39. CHAT VALIDATION

Send a message from Player 1.

Show the message appearing for Player 2.

Screenshot 35

Caption:

Figure 35: Real-time chat functionality between BINGO players

---

40. SECURITY IMPLEMENTATION

Security is integrated at multiple stages.

Source Code
↓
SonarQube
↓
OWASP Dependency Check
↓
Trivy FS
↓
Docker Image
↓
Trivy Image
↓
Deployment

This demonstrates the principle of **shifting security left**.

---

41. SECRETS MANAGEMENT

The following information must not be stored in GitHub:

AWS Access Keys
AWS Secret Keys
GitHub Tokens
Docker Hub Tokens
SonarQube Tokens
.pem Private Keys
Passwords
.env files containing secrets

Jenkins Credentials should be used instead.

Screenshot 36 – Jenkins Credentials

Show credential IDs/types but hide the secret values.

Caption:

Figure 36: Secure credentials configuration in Jenkins

---

42. TROUBLESHOOTING

Problem 1 — Jenkins unavailable

Check:

```
sudo systemctl status jenkins
```

Restart:

```
sudo systemctl restart jenkins
```

Problem 2 — Docker permission denied

Check:

```
docker ps
```

If Jenkins requires Docker access, ensure the Jenkins service account has the necessary Docker permissions and restart Jenkins.

Problem 3 — SonarQube unavailable

Check:

```
docker ps
```

Then:

```
docker logs sonar
```

Problem 4 — Application unavailable

Check:

```
docker ps
```

Then:

```
docker logs bingo-app
```

Problem 5 — Port 3000 unavailable

Check:

- Docker container
- EC2 Security Group
- Application port
- Public IP
- Docker port mapping

---

43. AWS COST MANAGEMENT

Because this project uses AWS, cost management is important.

The project should be monitored carefully even when using an AWS Free Tier account.

Check:

```
AWS Billing
   ↓
Cost Explorer
   ↓
Usage
```

When the project is no longer required:

```
terraform destroy
```

Also check for manually created resources.

44. TESTING

Create this table in your report.

Test Case	Description	Expected Result	Status
TC01	Open application	Application loads	PASS
TC02	Create room	Room created	PASS
TC03	Join room	Player joins	PASS
TC04	Multiple players	Players see same room	PASS
TC05	Number calling	Real-time update	PASS
TC06	Chat	Message received	PASS
TC07	Docker build	Image created	PASS
TC08	Docker run	Container runs	PASS
TC09	SonarQube	Analysis completed	PASS
TC10	Quality Gate	Result generated	PASS
TC11	OWASP	Dependency scan completed	PASS
TC12	Trivy FS	Filesystem scan completed	PASS
TC13	Docker Push	Image pushed	PASS
TC14	Trivy Image	Image scanned	PASS
TC15	Deployment	Application accessible	PASS
TC16	CI/CD	Pipeline successful	PASS

45. PROJECT RESULTS

The project successfully demonstrates:

Application

- Real-time BINGO
- Multiplayer rooms
- Real-time communication
- Chat
- BINGO board

Infrastructure

- AWS EC2
- Terraform provisioning
- Security Group configuration

CI/CD

- GitHub
- Jenkins
- Automated pipeline
- Docker build
- Docker push
- Deployment

DevSecOps

- SonarQube
- Quality Gate
- OWASP Dependency-Check
- Trivy filesystem scan
- Trivy image scan

---

46. ADVANTAGES

The proposed system provides:

1. Automated deployment
2. Repeatable infrastructure
3. Faster delivery
4. Better code-quality visibility
5. Automated vulnerability detection
6. Containerized deployment
7. Consistent environments
8. Reduced manual errors
9. Version-controlled infrastructure
10. Integrated security

---

47. LIMITATIONS

The current learning implementation has some limitations.

1. Game state may be stored in memory.
2. Restarting the application can remove active room state.
3. The architecture is not highly available.
4. A single EC2 instance can become a single point of failure.
5. HTTPS/TLS may not be configured.
6. No production database is required for the basic demonstration.
7. No automatic disaster recovery is implemented.
8. Kubernetes is not required for the basic implementation.

---

48. FUTURE ENHANCEMENTS

Future improvements could include:

Database

Use:

```
DynamoDB
```

or another persistent database.

Authentication

Add:

```
User Registration
Login
JWT/OAuth
Role-Based Access
```

HTTPS

Use:

```
Domain
  ↓
Load Balancer
  ↓
TLS Certificate
  ↓
Application
```

Monitoring

Add:

```
CloudWatch
Prometheus
Grafana
```

Kubernetes

The application could later be deployed using:

```
Docker
  ↓
EKS
  ↓
Kubernetes
```

when there is a genuine requirement for orchestration.

49. CONCLUSION

The Real-Time DevSecOps BINGO Application demonstrates how a modern web application can be developed, secured, containerized and deployed using an automated DevSecOps workflow.

The application uses Next.js, React and Socket.IO to provide real-time multiplayer functionality.

Terraform provides Infrastructure as Code for AWS resources.

Jenkins automates the CI/CD lifecycle.

SonarQube provides source-code quality analysis.

OWASP Dependency-Check identifies vulnerabilities in third-party dependencies.

Trivy provides filesystem and Docker image security scanning.

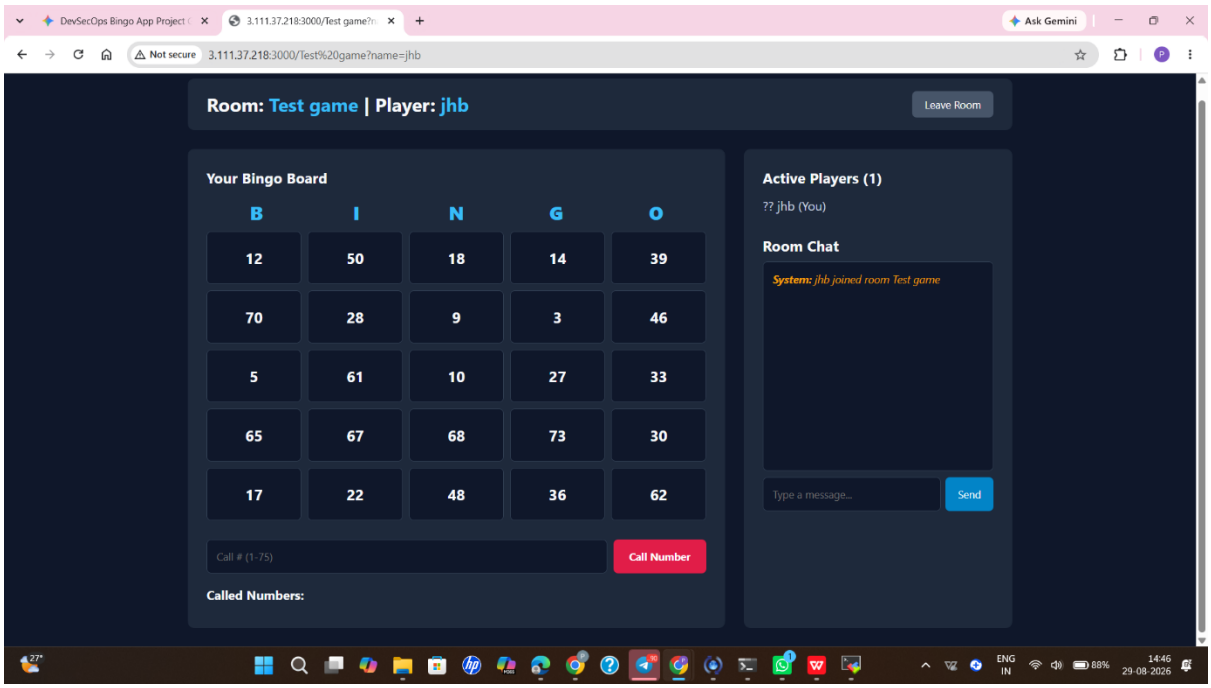
Docker provides application containerization, while Docker Hub acts as the container registry.

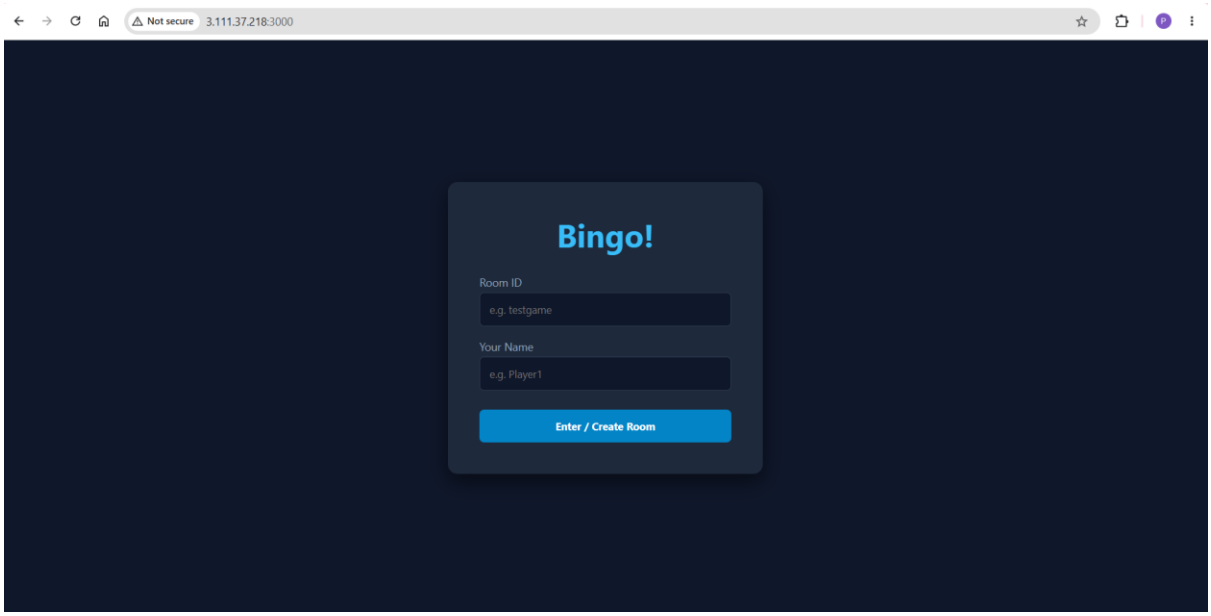
The final result is an automated workflow:

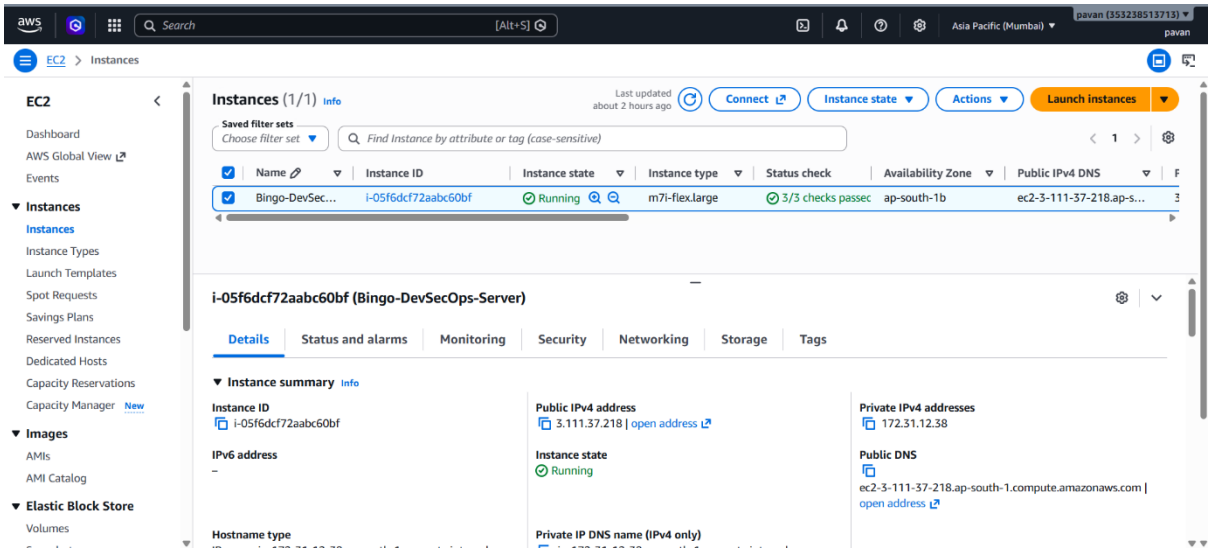
GitHub
↓
Jenkins
↓
SonarQube
↓
OWASP
↓
Trivy
↓
Docker
↓
Docker Hub
↓
Trivy Image Scan
↓
AWS EC2
↓
Live BINGO Application

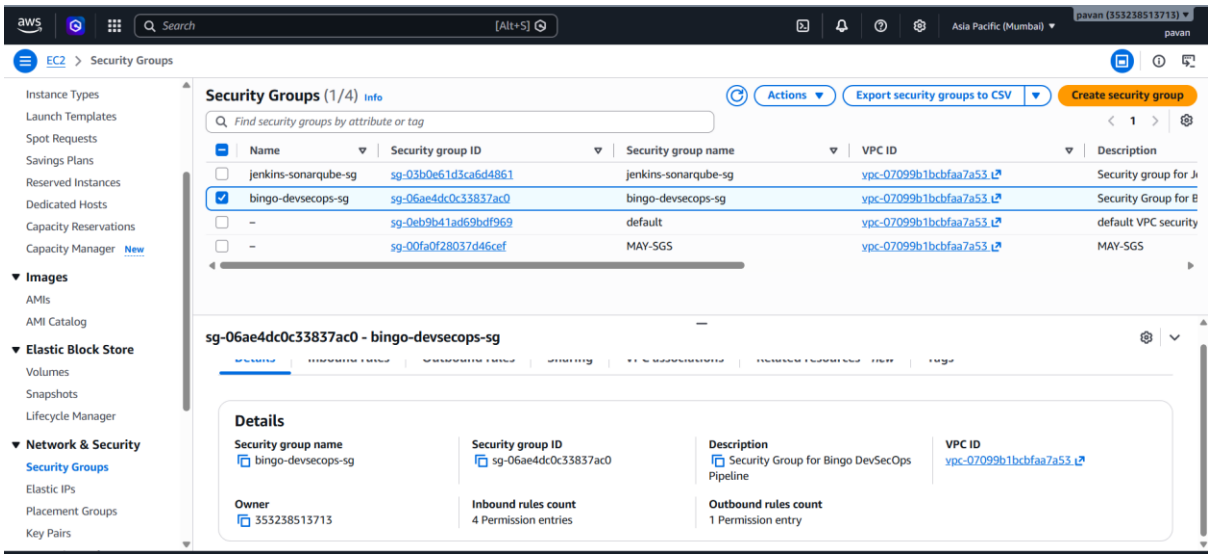
This project demonstrates the practical implementation of **DevSecOps, Infrastructure as Code, CI/CD, containerization, cloud deployment and application security**.

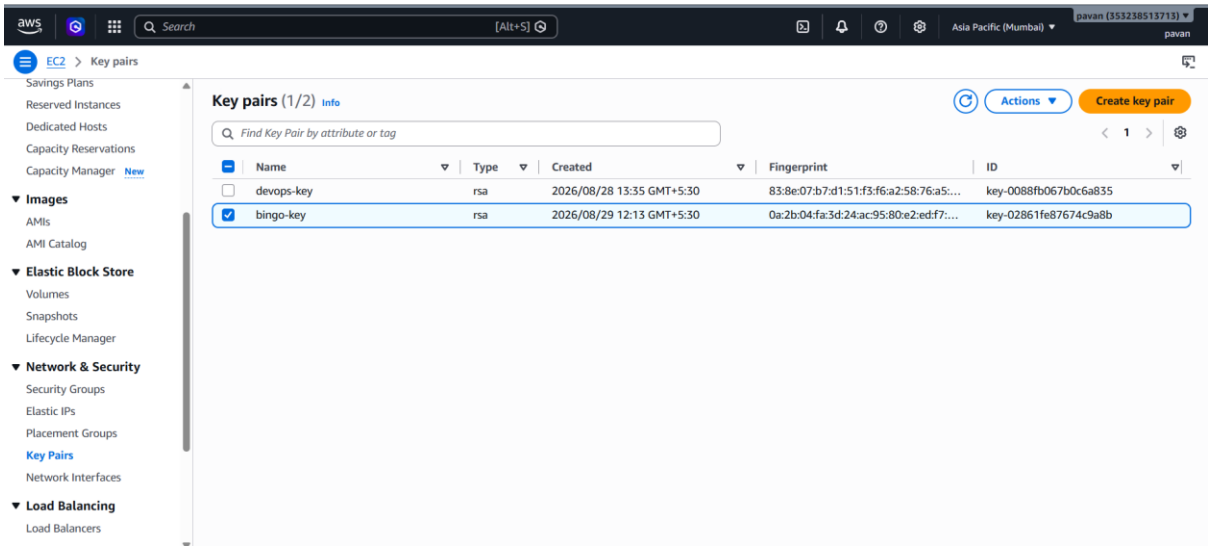
ScreenShots

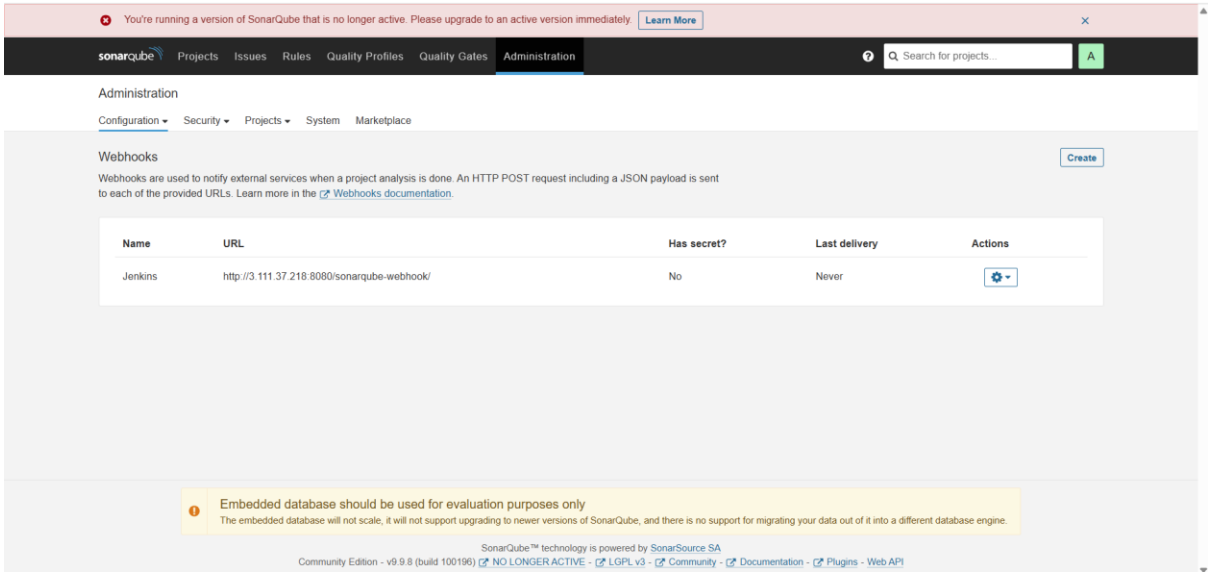


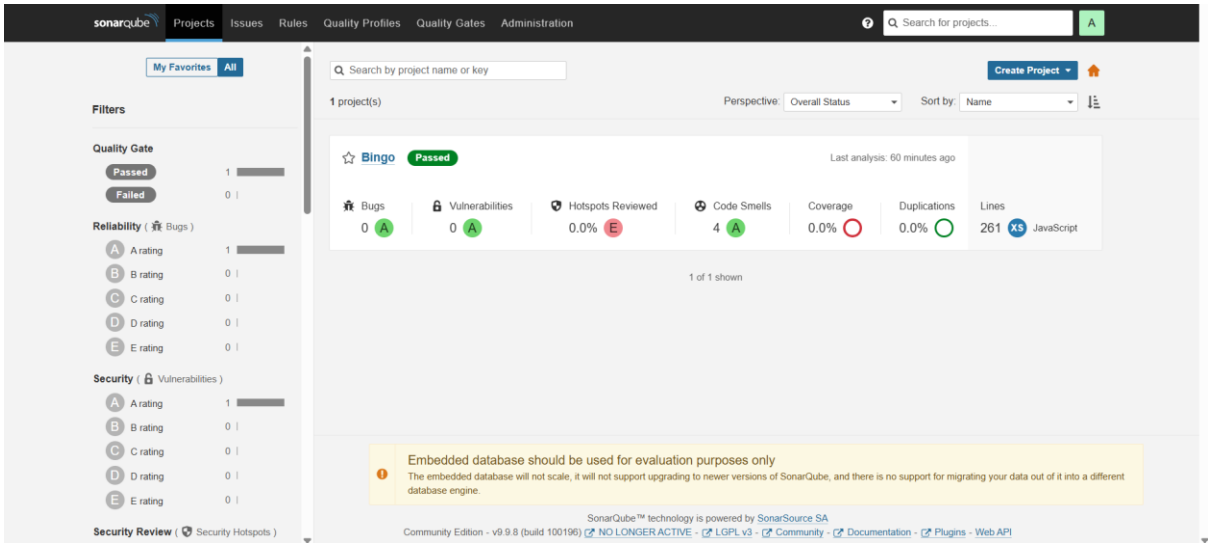


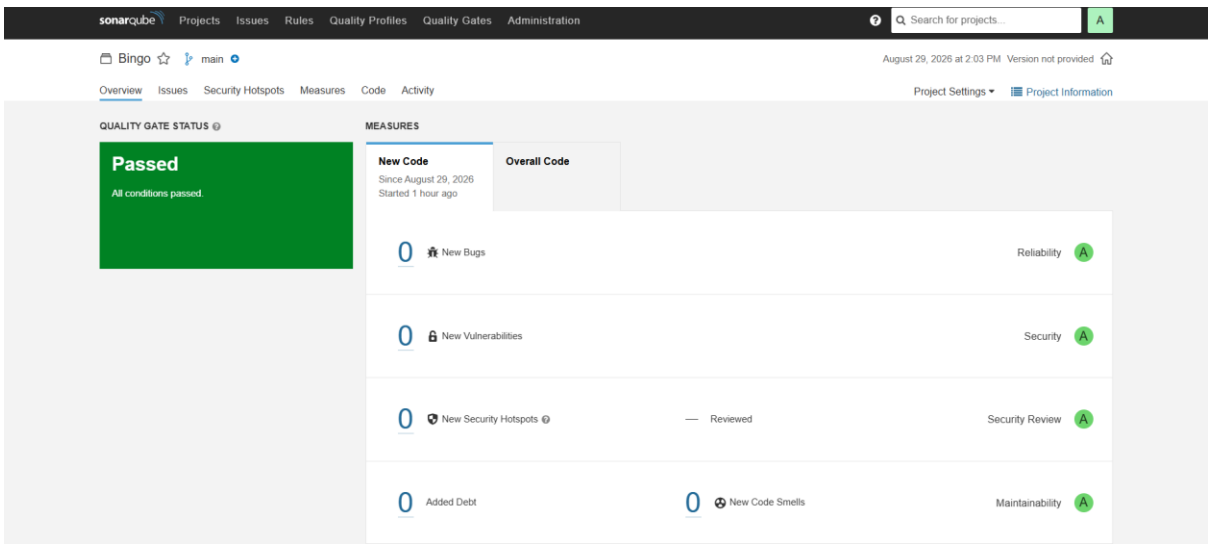

Search [Alt+S]
Asia Pacific (Mumbai)
pavan (353238513713)
pavan
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Instances (1/1) Info
Last updated about 2 hours ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Bingo-DevSec...
i-05f6dcf72aabc60bf
Running
m7i-flex.large
3/3 checks passed
ap-south-1b
ec2-3-111-37-218.ap-s...
i-05f6dcf72aabc60bf (Bingo-DevSecOps-Server)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-05f6dcf72aabc60bf
Public IPv4 address
3.111.37.218 | open address
Private IPv4 addresses
172.31.12.38
IPv6 address
–
Instance state
Running
Public DNS
ec2-3-111-37-218.ap-south-1.compute.amazonaws.com | open address
Hostname type
Private IP DNS name (IPv4 only)


Search [Alt+S]
Asia Pacific (Mumbai)
pavan (353238513713)
pavan
EC2 > Security Groups
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Security Groups (1/4) Info
Actions
Export security groups to CSV
Create security group
Find security groups by attribute or tag
Name
Security group ID
Security group name
VPC ID
Description
jenkins-sonarqube-sg
sg-03b0e61d3ca6d4861
jenkins-sonarqube-sg
vpc-07099b1bcbfaa7a53
Security group for J
bingo-devsecops-sg
sg-06ae4dc0c33837ac0
bingo-devsecops-sg
vpc-07099b1bcbfaa7a53
Security Group for B
–
sg-0eb9b41ad69bdf969
default
vpc-07099b1bcbfaa7a53
default VPC security
–
sg-00fa0f28037d46cef
MAY-SGS
vpc-07099b1bcbfaa7a53
MAY-SGS
sg-06ae4dc0c33837ac0 - bingo-devsecops-sg
Details
Security group name
bingo-devsecops-sg
Security group ID
sg-06ae4dc0c33837ac0
Description
Security Group for Bingo DevSecOps Pipeline
VPC ID
vpc-07099b1bcbfaa7a53
Owner
353238513713
Inbound rules count
4 Permission entries
Outbound rules count
1 Permission entry

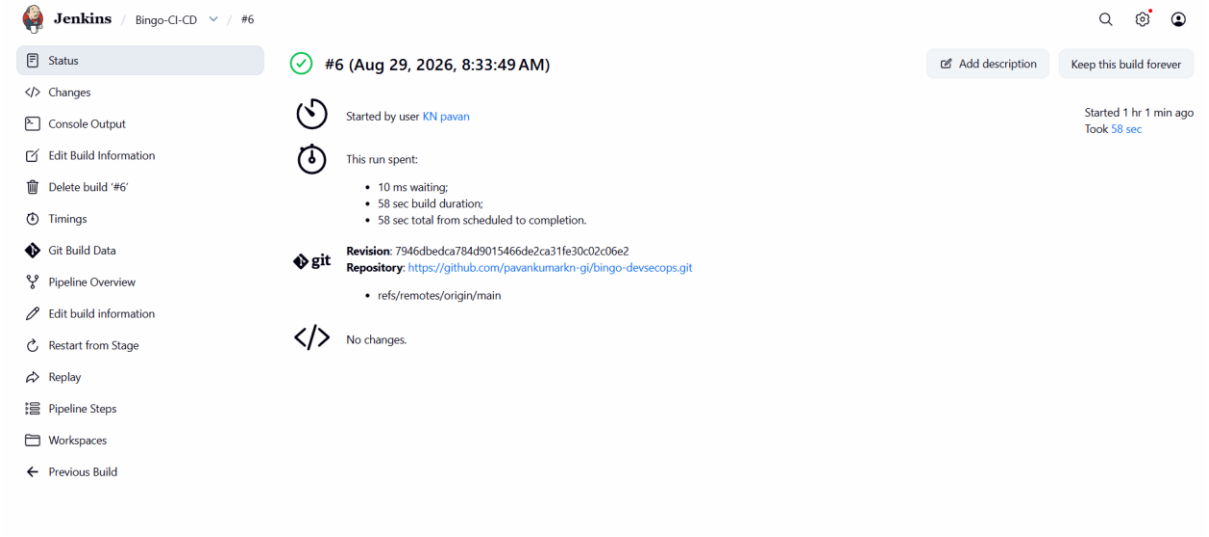

Search [Alt+S]
Asia Pacific (Mumbai)
pavan (353238513713)
pavan
EC2 > Key pairs
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Key pairs (1/2) Info
Actions
Create key pair
Find Key Pair by attribute or tag
Name
Type
Created
Fingerprint
ID
devops-key
rsa
2026/08/28 13:35 GMT+5:30
83:8e:07:b7:d1:51:f3:f6:a2:58:76:a5:...
key-0088fb067b0c6a835
bingo-key
rsa
2026/08/29 12:13 GMT+5:30
0a:2b:04:fa:3d:24:ac:95:80:e2:ed:f7:...
key-02861fe87674c9a8b

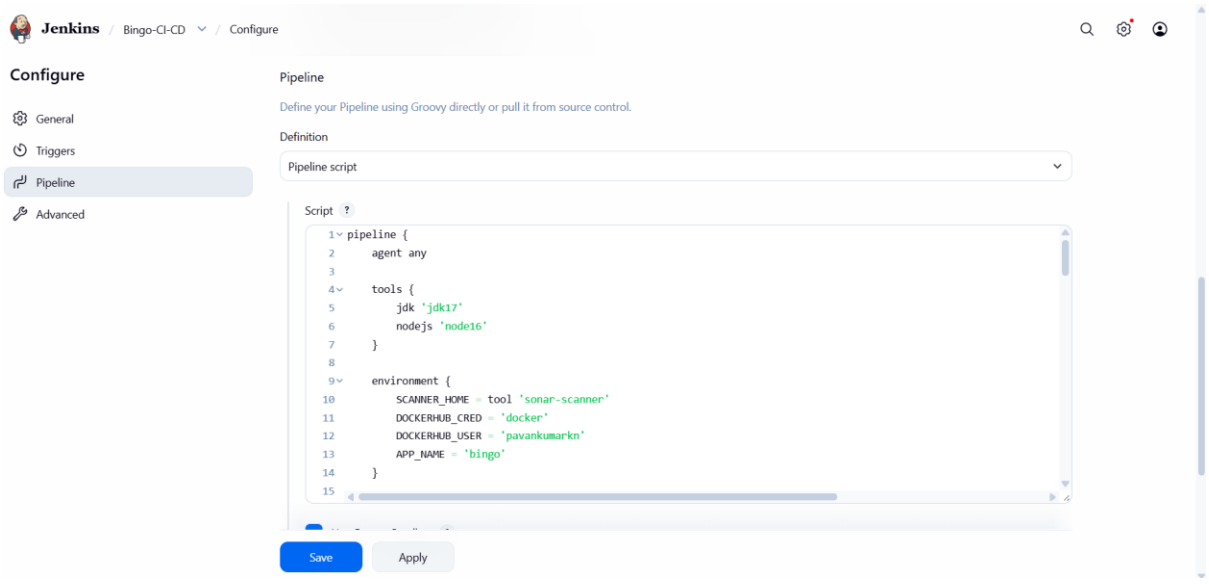

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. Learn More
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
Administration
Configuration Security Projects System Marketplace
Webhooks
Create
Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the Webhooks documentation.
Name URL Has secret? Last delivery Actions
Jenkins http://3.111.37.218:8080/sonarqube-webhook/ No Never
Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.
SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API

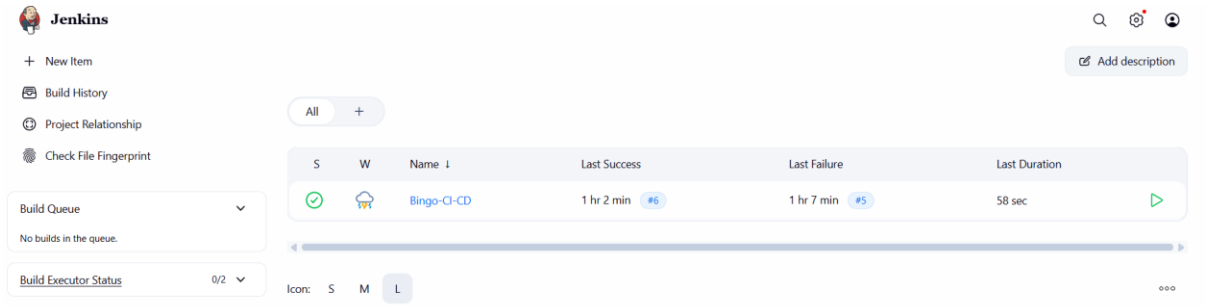

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
My Favorites All
Filters
Quality Gate
Passed 1
Failed 0
Reliability (Bugs)
A rating 1
B rating 0
C rating 0
D rating 0
E rating 0
Security (Vulnerabilities)
A rating 1
B rating 0
C rating 0
D rating 0
E rating 0
Security Review (Security Hotspots)
Search by project name or key
Create Project
1 project(s)
Perspective: Overall Status
Sort by: Name
Bingo Passed
Last analysis: 60 minutes ago
Bugs 0 A
Vulnerabilities 0 A
Hotspots Reviewed 0.0% E
Code Smells 4 A
Coverage 0.0%
Duplications 0.0%
Lines 261 XS JavaScript
1 of 1 shown
Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.
SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API

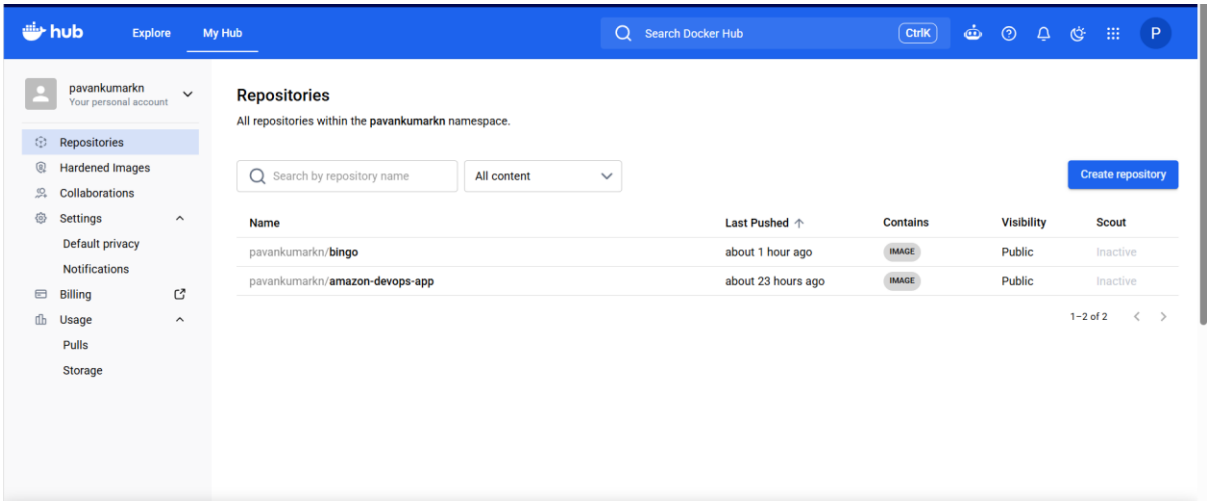

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
Bingo main
August 29, 2026 at 2:03 PM Version not provided
Overview Issues Security Hotspots Measures Code Activity
Project Settings Project Information
QUALITY GATE STATUS
Passed
All conditions passed.
MEASURES
New Code
Since August 29, 2026
Started 1 hour ago
Overall Code
0 New Bugs
Reliability A
0 New Vulnerabilities
Security A
0 New Security Hotspots
Reviewed
Security Review A
0 Added Debt
0 New Code Smells
Maintainability A

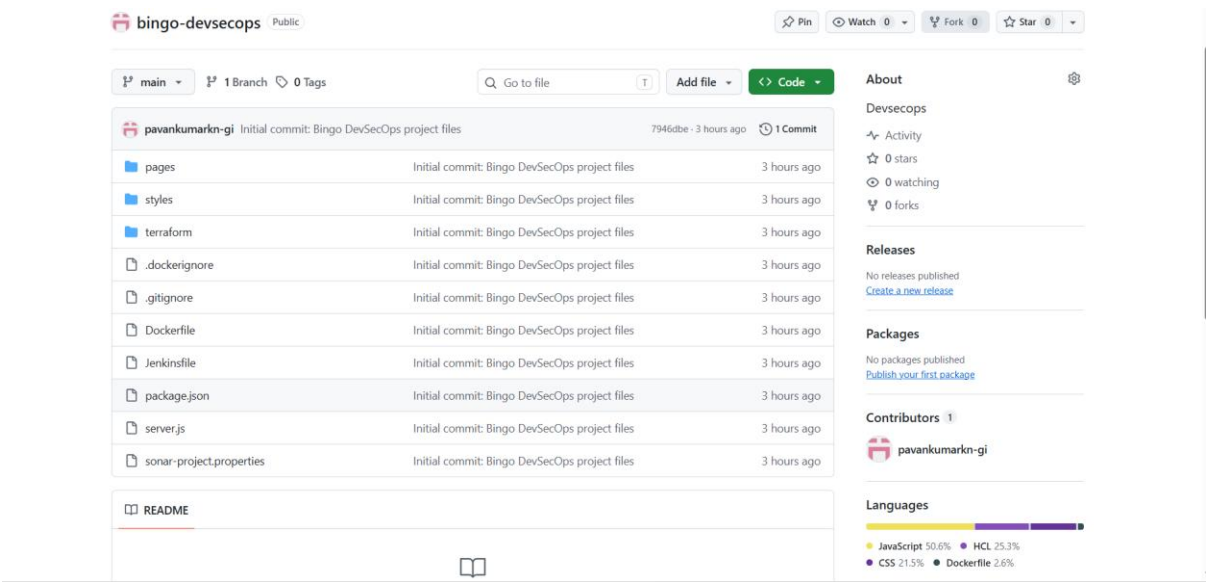

Jenkins / Bingo-CI-CD
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Full Stage View
SonarQube
Stages
Pipeline Syntax
Builds
Filter
Today
#6 8:33 AM
#5 8:29 AM
#4 8:27 AM
Bingo-CI-CD
Add description
Stage View
Declarative: Tool Install
Clean Workspace
Git Checkout
SonarQube Analysis
Quality Gate Check
Trivy Filesystem Scan
Docker Build & Tag
Trivy Image Scan
Docker Push to Hub
Deploy Container
Average stage times:
(full run time: ~58s)
173ms
302ms
1s
17s
347ms
529ms
14s
2s
7s
12s
Aug 29
14:03
No Changes
SonarQube Quality Gate
Bingo
Passed
server-side processing:
Success
Permalinks
Last build (#6), 1 hr 0 min ago
Last stable build (#6), 1 hr 0 min ago
Last successful build (#6), 1 hr 0 min ago
Last failed build (#5), 1 hr 5 min ago


Jenkins / Bingo-CI-CD / #6
Status
Changes
Console Output
Edit Build Information
Delete build '#6'
Timings
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
#6 (Aug 29, 2026, 8:33:49 AM)
Add description
Keep this build forever
Started by user KN pavan
Started 1 hr 1 min ago
Took 58 sec
This run spent:
10 ms waiting;
58 sec build duration;
58 sec total from scheduled to completion.
Revision: 7946dbedca784d9015466de2ca31fe30c02c06e2
Repository: https://github.com/pavankumarkn-gi/bingo-devsecops.git
refs/remotes/origin/main
No changes.


Jenkins / Bingo-CI-CD / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
tools {
jdk 'jdk17'
nodejs 'node16'
}
environment {
SCANNER_HOME = tool 'sonar-scanner'
DOCKERHUB_CRED = 'docker'
DOCKERHUB_USER = 'pavankumarkn'
APP_NAME = 'bingo'
}
Save
Apply


Jenkins
New Item
Build History
Project Relationship
Check File Fingerprint
Add description
All
S
W
Name ↓
Last Success
Last Failure
Last Duration
Bingo-CI-CD
1 hr 2 min #6
1 hr 7 min #5
58 sec
Build Queue
No builds in the queue.
Build Executor Status 0/2
Icon: S M L


hub
Explore
My Hub
Search Docker Hub
CtrlK
pavankumarkn
Your personal account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Repositories
All repositories within the pavankumarkn namespace.
Search by repository name
All content
Create repository
Name
Last Pushed
Contains
Visibility
Scout
pavankumarkn/bingo
about 1 hour ago
IMAGE
Public
Inactive
pavankumarkn/amazon-devops-app
about 23 hours ago
IMAGE
Public
Inactive
1–2 of 2


bingo-devsecops
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
pavankumarkn-gi Initial commit: Bingo DevSecOps project files
7946dbe · 3 hours ago
1 Commit
pages
styles
terraform
.dockerignore
.gitignore
Dockerfile
Jenkinsfile
package.json
server.js
sonar-project.properties
Initial commit: Bingo DevSecOps project files
3 hours ago
README
About
Devsecops
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
pavankumarkn-gi
Languages
JavaScript 50.6%
HCL 25.3%
CSS 21.5%
Dockerfile 2.6%

THANK YOU